

Writeup: – Challenge "Lingagu"

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup_09_lingaguOK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `KNT$3ENRA?pHJ6#x`

Résumé

Premiers réflexes :

```
file Lingagu
```

Outils Utilisés

Lingagu: ELF 64-bit LSB executable, x86-64 ...

```
chmod +x Lingagu      #au cas ou
```

```
strings -n4 Lingagu | less
```

Analyse Statique

Strings intéressants :

Good Job!

Bad Password!

state0

state1

...

state15

```
objdump -d -M intel Lingagu > disas.txt  
less disas.txt
```

Ce qu'on repère très vite :

La fonction `_start` lit 0x64 octets (`sys_read`) dans un buffer 0x402018.

Elle demande une longueur de 0x10 (16) octets (`cmp r8,0x10`) sinon elle saute à `bad_label`.

Puis on voit une boucle d'états `state0` → `state1` → ... → `state15`, chacun compare un octet calculé (`movzx ebx, bl; cmp BYTE PTR [rsi+rcx], bl`).

⇒ On a donc 16 transformations successives, chaque fois :

`rbx` est modifié par quelques opérations arithmétiques / logiques ;

on ne compare que l'octet de poids faible (bl) ;

rbx est ensuite écrasé par ce même octet (zero-extend) (movzx)

la résolution via python :

On reproduit en Python les 16 fonctions (ici nommées state0 ... state15) et on enchaîne.

!/usr/bin/env python3

solve_lingagu.py

MASK = 0xFFFFFFFFFFFFFFFF

```
def rol(x,n,bits=64): return ((x<<n)|(x>>(bits-n))) & ((1<<bits)-1)
```

```
def state0(r): r=((r*0x17)<<2 & MASK)-0x11; return r&0xFF
```

```
def state1(r): r=(r*2 & MASK)-0x48; return r&0xFF
```

```
def state2(r): r=(r ^0x1A) & MASK; return r&0xFF
```

```
def state3(r): r=(-r & MASK)-0x88; return r&0xFF
```

```
def state4(r): r=((r<<1)&MASK)-0x15; return r&0xFF
```

```
def state5(r): r=(rol(r,1)-0x21) & MASK; return r&0xFF
```

```
def state6(r): r=(r*2-0x3C) & MASK; return r&0xFF
```

```
def state7(r): r=((r|0x10)-0x0C) & MASK; return r&0xFF
```

```
def state8(r): r=((r|1)&0xED) & MASK; return r&0xFF
```

```
def state9(r): r=(r+4-6) & MASK; return r&0xFF
```

```
def state10(r): r=((r<<1)-0x0E)&MASK; return r&0xFF
```

```
def state11(r): r=((r>>1)+0x10)&MASK; return r&0xFF
```

```
def state12(r): r=(r+2) & MASK; return r&0xFF
```

```
def state13(r): r=(r-0x1E+0x0A)&MASK; return r&0xFF
```

```
def state14(r): r=((r>>1)+0x08)&MASK; return r&0xFF
```

```
def state15(r): r=(r*5-0x37)&MASK; return r&0xFF
```

```
states = [state0,state1,state2,state3,state4,
state5,state6,state7,state8,state9,
state10,state11,state12,state13,state14,state15]
```

```
rbx = 1
```

```
flag = []
```

```
for f in states:
```

```
    c = f(rbx)
```

```
    flag.append(chr(c))
```

```
    rbx = c      #zero-extend bl → rbx
```

```
print(''.join(flag))
```

Analyse Dynamique

L'exécution du programme a permis d'observer son comportement et d'identifier les points de validation.

Identification du Mécanisme de Validation

Puis on voit une boucle d'états `state0 → state1 → ... → state15`, chacun compare un octet calculé (`movzx ebx, bl; cmp BYTE PTR [rsi+rcx], bl`).

⇒ On a donc 16 transformations successives, chaque fois :

Découverte du Flag

Le bon flag trouvé : `KNT$3ENRA?pHJ6#x`

Conclusion

Ce crackme a été résolu en comprenant son mécanisme de validation et en élaborant une stratégie appropriée.